

Pacman

ゴースト！もっとゴースト！

概要：有名なアーケードゲームPac-manを再現しましょう！

バージョン：1.5

目次

I はじめに **2**

II AIに関する指示 **3**

III 共通指示 **5**
- III.1 一般規則 5
- III.2 Makefile 5
- III.3 追加指針 6

IV 必須部分 **7**

V 詳細仕様 **8**
- V.1 使用方法 8
- V.2 設定ファイル 8
- V.3 不正な設定への対処 9
- V.4 迷路生成器の統合 9
- V.5 ハイスコアシステム 9

VI ゲーム仕様 **11**
- VI.1 レベルの構造 11
- VI.2 プレイヤー 12
- VI.3 ゴースト 12
- VI.4 パックガムとスーパーパックガム 12
- VI.5 チートモード 13
- VI.6 得点 14
- VI.7 ゲームの進行 14
- VI.8 ユーザーインターフェース 15

VII プロジェクトのパッケージ化 **16**

VIII プロジェクト管理 **17**

IX README の要件 **18**

X 提出と相互レビュー **20**

第 I 章

はじめに

1980年にナムコから初めて発売されたPac-Manは、たちまち文化的な象徴となり、史上最も影響力のあるビデオゲームの1つになりました。岩谷徹によって設計され、当時の宇宙シューティングゲームとは異なり、女性や気軽に遊ぶプレイヤーにも楽しめるゲームを目指していました。このゲームは、今では有名なゴーストAIを導入しました。それぞれ固有の行動を持ちます（Blinkyは追跡、Pinkyは待ち伏せ、Inkyは予測不能、Clydeは……奇妙です）。

Pac-Manは、ゴーストを食べられるようにするパックガム（パワーエサ）という、パワーアップの概念を広めた最初のゲームでもありました。元のアーケード版には256のレベルがありましたが、整数オーバーフローの不具合によってレベル256はクリア不可能で、悪名高い「キルスクリーン」として知られています。

このプロジェクトでは、現代的な構造とプロジェクト構成を備え、実際のゲーム配信プラットフォームに展開できる独自のPython版を作り、この名作に新たな命を吹き込みます。ワカワカ！

第 II 章

AI に関する指示

背景

学習の過程で、AI はさまざまな作業を支援できます。時間をかけて AI ツールの多様な機能と、それらが作業をどう支援できるかを調べてください。ただし、常に慎重に扱い、結果を批判的に検討してください。コード、文書、アイデア、技術的な説明のいずれでも、自分の質問が適切だったか、生成内容が正確かを完全に確信することはできません。仲間は、間違いや見落としを避けるための貴重な存在です。

主なメッセージ

- AI を使って反復的で退屈な作業を減らしてください。
- コーディングとそれ以外の両方で、将来の仕事に役立つプロンプト作成能力を育ててください。
- AI システムの仕組みを学び、一般的なリスク、偏り、倫理上の問題を予測して避けられるようにしてください。
- 仲間と作業し、技術力と対人・行動面の能力の両方を伸ばし続けてください。
- 完全に理解し、責任を持てる AI 生成物だけを使用してください。

学習者のルール

- AI ツールを調べて仕組みを理解する時間を取り、倫理的に利用し、潜在的な偏りを減らせるようにする。
- プロンプトを送る前に問題をよく考える。これにより、正確な用語を使い、明確で詳しく、目的に合ったプロンプトを書ける。
- AI が生成したものを、体系的に確認、レビュー、疑問視、テストする習慣を付ける。
- 常にピアレビューを求める。自分だけの確認に頼らない。

この段階で得られる成果

- 汎用的なプロンプト作成能力と、専門分野に特化した能力の両方を伸ばす。
- AI ツールを効果的に使い、生産性を高める。
- 計算論的思考、問題解決、適応力、協働を引き続き強化する。

補足と例

- 試験や評価など、本当の理解を示さなければならない場面は繰り返し訪れます。備えを怠らず、技術力と対人能力の両方を伸ばしてください。
- 考え方を説明し、仲間と議論すると、理解の抜けが見つかることがよくあります。仲間との学習を優先してください。
- AI ツールはあなた固有の状況を知らず、一般的な回答を返す傾向があります。同じ環境を共有する仲間は、より適切で正確な洞察を提供できます。
- AI が最もありそうな答えを生成しがちな一方、仲間は別の視点や重要な細かな違いを示せます。品質を確認する存在として頼ってください。

✓ よい実践例

> AI に「ソート関数はどうテストすればよい？」と尋ね、いくつかの案を得ます。それを試し、仲間と結果を確認して、一緒に方法を改善します。

✗ 悪い実践例

> AI に関数全体を書かせ、そのままプロジェクトに貼り付けます。ピア評価では、その関数の動作や理由を説明できません。信頼を失い、プロジェクトも不合格になります。

✓ よい実践例

> パーサーの設計を AI に手伝ってもらいます。その後、仲間とロジックを順に確認します。二つのバグを見つけ、一緒に書き直し、よりよく整理された、完全に理解できるものにします。

✗ 悪い実践例

> プロジェクトの重要部分を Copilot に生成させます。コンパイルはできますが、パイプをどう扱うのか説明できません。評価で妥当性を説明できず、プロジェクトは不合格になります。

第 III 章

共通指示

III.1 一般規則

- Python 3.10以降で記述してください。
- flake8のコーディング規約に従ってください。
- クラッシュを避けるため、関数は例外を適切に処理してください。起こり得るエラーはtry-exceptブロックで処理します。ファイルや接続などのリソースには、自動的な後始末を確実にするコンテキストマネージャーを優先してください。レビュー中、未処理の例外でプログラムがクラッシュした場合は、機能していないと見なします。
- すべてのリソース（ファイルハンドル、ネットワーク接続など）を適切に管理し、リークを防いでください。可能ならコンテキストマネージャーで自動管理してください。
- 関数の引数、戻り値、および該当する変数に、typingモジュールを使って型ヒントを付けてください。静的型検査にはmypyを使います。すべての関数がmypyをエラーなしで通過する必要があります。
- PEP 257に従うdocstring（Google形式やNumPy形式など）を関数とクラスに記述し、目的、引数、戻り値を説明してください。

III.2 Makefile

一般的な作業を自動化するMakefileをプロジェクトに含めてください。次のルールが必要です（必須のlintでは指定フラグが必要です。より厳密な検査のため-strictを試すことを強く推奨します）：

- install：pip、uv、pipx、その他任意のパッケージマネージャーで依存関係をインストールする。
- run：プロジェクトのメインスクリプトを実行する（例えばPythonインタープリタ経由）。
- debug：Python組み込みのデバッガー（pdbなど）を使い、メインスクリプトをデバッグモードで実行する。
- clean：一時ファイルやキャッシュ（__pycache__、.mypy_cacheなど）を削除し、プロジェクト環境をきれいに保つ。

- lint：flake8 .とmypy . --warn-return-any --warn-unused-ignores --ignore-missing-imports --disallow-untyped-defs --check-untyped-defsを実行する

- lint-strict（任意）：flake8 .とmypy . --strictを実行する

III.3 追加指針

- プロジェクトの機能を検証するテストプログラムを作成してください（提出・採点の対象外）。pytestやunittestなどのフレームワークで、境界的なケースも含む単体テストを行ってください。
- Pythonの生成物を除外する.gitignoreファイルを含めてください。
- 開発中の依存関係を分離するため、仮想環境（venvやcondaなど）の使用を推奨します。

プロジェクト固有の追加要件があれば、この節の直後に記載します。

第 IV 章

必須部分

このプロジェクトでは、オブジェクト指向プログラミング、単純なグラフィックスライブラリ（MLXなど）、モジュール化された再利用可能な設計を使い、完全に遊べるPac-ManゲームをPythonで作成します。



使用するすべての関数にMLXライブラリ内で同等の関数があれば、そのグラフィックスライブラリをMLXと同等と見なします。使用する関数にMLXで同等のものがなければ、その関数は使用できません。

ゲームは次をサポートしてください：

- ファイル（コメント付きJSON）でゲームのパラメータを変更する設定。
- 堅牢なエラー処理。クラッシュ不可！
- 外部のA-Maze-ingパッケージに基づくレベル生成（自分たちのパッケージではありません！）。
- 任意の方法による永続的なハイスコアシステム（プロジェクト内のjsonファイルやディスクへの保存など）。
- メインメニュー、ゲーム画面、ゲームオーバー処理を備える、洗練されたグラフィカルUI。
- 評価用のチートモード。
- 実演のための公開ゲーム配信プラットフォーム（Steam、Itch.ioなど）への展開。

プロジェクト管理を特に重視し、複数の文書の提出を求めます。

ゲームループ

- メインメニュー > ゲーム開始 > 勝利または敗北 > ハイスコア用の名前入力 > メインメニューに戻る

第 V 章

詳細仕様

V.1 使用方法

プログラムはコマンドラインから次のように起動してください：

```
$> python3 pac-man.py config.json
```

- 引数は設定ファイル1つだけを受け取ってください。ファイル名は自由ですが、jsonファイルである必要があります。
- あらゆるエラー（ファイル不足、不正な値、キー不足）は明確なメッセージで適切に処理し、Pythonのトレースバックを表示してはいけません。

V.2 設定ファイル

設定ファイルはJSONを使用します。標準のJSON形式に加え、コメントを処理してください。#で始まる行はコメントとして無視します。追加のコメント形式（CやC++形式など）に対応してもかまいません。
具体的な構造は自由ですが、キーをREADMEに記載し、安全に使える既定値を用意してください。

推奨するキー（名前は例示）：

- `highscore_filename`
- `level`：複数のレベルを含む配列
- 各レベルのwidth、height
- `lives`：3
- `pacgum`：42
- `points_per_pacgum`：10
- `points_per_super_pacgum`：50
- `points_per_ghost`：200

- `seed : 42`
- `level_max_time : 90`

V.3 不正な設定への対処

値が不足している、または不正な場合は、安全な既定値に補正し、明確なメッセージをログに記録して続行してください。不明なキーは無視してください。トレースバックは表示しません。



口頭評価中にゲームの設定を変更します。

V.4 迷路生成器の統合

自分で生成器を書いてはいけません。
プロジェクト開始時、別のグループのA-Maze-ingパッケージが割り当てられます：

- そのパッケージを変更せず、そのまま使用してください。割り当てられたパッケージは相互レビュー中に再インストールします。
- ローダーを相手のインターフェースに合わせてください。逆ではありません。
- Pac-Manに適した通路を生成するため、パラメータPERFECTをFalseに設定します（A-Maze-ingの課題文を参照）。
- 生成器が失敗した場合は、エラーを適切に処理してください。

V.5 ハイスコアシステム

永続的なハイスコアシステムを実装してください。
具体的な実装は自由ですが、READMEに説明してください。

次の要件を満たしてください：

- 任意の方法で保存する（プロジェクト内のjsonファイル、ディスクなど）。
- ファイルのエラー（ファイル不足、不正な形式など）に耐える。
- プレイヤー名を扱う（最大10文字、英数字とスペースのみ）。
- 得点を扱う（非負の整数）。

- 上位10件のハイスコアをプレイヤー名と得点とともに保持する。
- ゲーム開始時にハイスコアを読み込み、終了時に保存する。
- ゲーム終了時（勝利・敗北のいずれでも）に、プレイヤーが名前を入力できる。
- メインメニューにハイスコアを表示する。

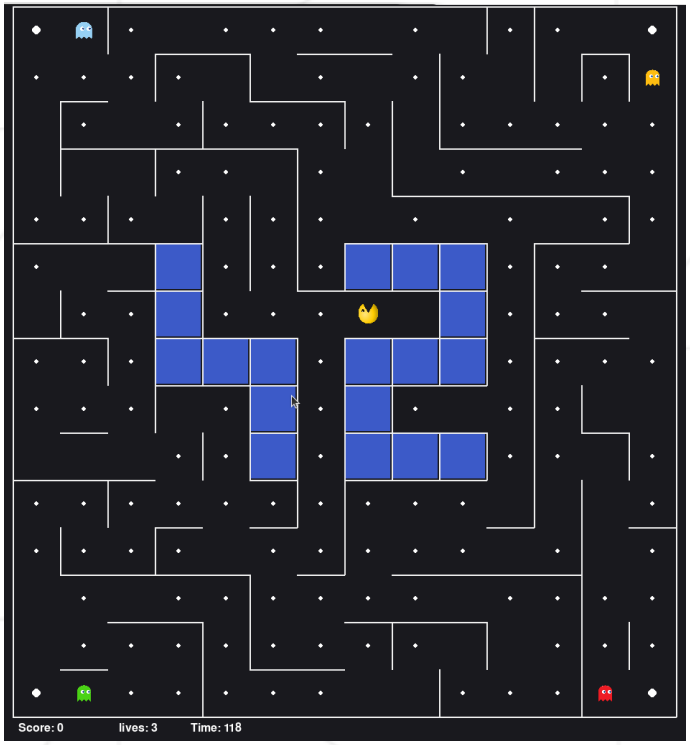


第 VI 章

ゲーム仕様

VI.1 レベルの構造

- 割り当てられたA-Maze-ingパッケージが生成する、壁と通路を持つ迷路。
- 最初のレベルは固定シード（例：42）で生成した迷路で構成し、それ以降の各レベルはランダムに生成した迷路で構成する。
- 大半の通路にパックガム（小さな点）。
- 迷路の4隅にスーパーパックガム（パワーエサ）。
- ゴーストは4体で、迷路の各隅に1体ずつ。
- プレイヤーは迷路の中央から開始する。



VI.2 プレイヤー

- 通路だけを移動できる（壁は通れない）。
- 矢印キーまたはWASD（キーボードに応じて）で、上下左右の4方向に移動できる。
- 残機3で開始する。
- ゴーストに触れると残機を1失う。
- 残機を失うと迷路の中央で復活する。
- 残機をすべて失うとゲームオーバーになる。
- パックガムをすべて食べるとレベルをクリアする。
- 全レベルをクリアするとゲームに勝利する。
- パックガムを食べると得点がX点増える。
- スーパーパックガム（パワーエサ）を食べると得点がY点増え、短時間ゴーストを食べられるようになる。
- 食べられる状態のゴーストを食べると得点がZ点増える。

VI.3 ゴースト

- 通路を自律的に移動する。
- 食べられない状態ではプレイヤーを追跡する。追跡の動作は自由に定義できる（距離に基づく、ランダムなど）。
- 食べられる状態ではプレイヤーから逃げる。
- 食べられたら、しばらくして自分の隅に復活する（例えば5秒後や10秒後）。

VI.4 パックガムとスーパーパックガム

- パックガムは、大半の通路に配置する小さな点。
- スーパーパックガム（パワーエサ）は、迷路の4隅に配置する大きな点。
- パックガムを食べると得点がX点増える。
- スーパーパックガムを食べると得点がY点増え、短時間ゴーストを食べられるようになる。

VI.5 チートモード

- 相互レビューを容易にするため、有効にできる。
- 推奨する機能：
    - 無敵（残機が減らず、ゴーストがプレイヤーを食べられない）。
    - レベルスキップ（現在のレベルを即座にクリアする）。
    - ゴースト停止（ゴーストが動かなくなる）。
    - 残機追加（プレイヤーの残機を増やす）。
    - 速度上昇（プレイヤーの移動速度が上がる）。
    - その他、役立つ機能。

チートモードは相互レビューのためのものです。評価者がゲームの全機能を簡単にテストできるよう、実際に役立つものにする必要があります。

VI.6 得点

- 次の場合に得点が増える：
  - パックガムを食べる（+X点）。
  - スーパーパックガムを食べる（+Y点）。
  - 食べられる状態のゴーストを食べる（+Z点）。
- 得点は減らない。

VI.7 ゲームの進行

- ゲームは複数のレベル（少なくとも10）で構成する。
- 各レベルには制限時間がある（例：90秒）。
- 制限時間に達したときの動作は自由に決められる（レベルの再開始、ゲーム終了など）。
- レベルをクリアすると次のレベルに進む。
- レベルを移る際、得点と残機を引き継ぐ。
- 全レベルをクリアするか、残機をすべて失うとゲームが終了する。
- ゲーム中、プレイヤーは一時停止と再開ができる。
- ゲーム終了時（勝利・敗北のいずれでも）に最終得点を表示し、プレイヤーが名前を入力してハイスコアを保存できる。
- ゲーム終了後、メインメニューに戻る。

VI.8 ユーザーインターフェース

- メインメニュー：
  - ゲーム開始：新しいゲームを開始できる。
  - ハイスコア表示：上位10件の得点とプレイヤー名を表示する。
  - 遊び方：操作方法と規則を表示する。
  - 終了：ゲームアプリケーションを閉じる。
- ゲーム中のHUD（プレイ中は常に表示）：
  - 現在の得点
  - 残機
  - 現在のレベル
  - レベルの残り時間
- 一時停止メニュー：
  - ゲームを再開：進行中のゲームに戻れる。
  - メインメニューに戻る：現在のゲームを終了し、メインメニューに戻れる。
- ゲームオーバー画面：
  - 最終得点を表示する。
  - ハイスコア一覧に得点を保存するため、プレイヤーに名前の入力を促す。
- 勝利画面：
  - 最終得点と祝福のメッセージを表示する。
  - ハイスコア一覧に得点を保存するため、プレイヤーに名前の入力を促す。

第 VII 章

プロジェクトのパッケージ化

次を提供してください：

- 公開プラットフォーム（SteamやItch.ioなど）から、無料の限定公開または非公開ビルドとしてインストール・起動できる完全なゲーム。
- パッケージ化したゲームは、完全に動作する必要があります。
- パッケージ内に最低限の説明（操作、オプション、設定）を含めてください。
- Gitリポジトリには全ソースコードを含め、ルートにパッケージ作成用スクリプトまたは仕様を置いてください。
- 相互レビュー中にパッケージの再生成を求められる場合があります。

第VIII章

プロジェクト管理

このプロジェクトでは、体系的な方法で進行を管理し、その証拠を提供してください。プロジェクト管理を支える方法やツールは自由に選べます。採用した方法を裏付けるさまざまな文書を作成してください。リポジトリ内の専用サブディレクトリに、関連する資料を含めてください。

例：

- プロジェクトのスケジュール、ガントチャート、カンバンなど。
- 計画と比較した実際の進捗の追跡。
- プロジェクト分析と、それに伴う選択。
- リスク分析と、考えられる緩和策。
- チームの体制（誰が何を担当したか、意思決定の方法、問題への対処方法）。
- 受け入れテスト計画（テストした機能、発見・修正した不具合）。
- プロジェクト中に生じた障害や対立の概要。
- その他。

第IX章

README の要件

Git リポジトリのルートに README.md を用意してください。プロジェクトを知らない人（仲間、スタッフ、採用担当者など）が、内容、実行方法、詳しい情報の入手先を素早く理解できるようにするためです。
README.md には最低限、以下を含めてください。

- 最初の行はイタリックにし、次の文章にしてください：This project has been created as part of the 42 curriculum by <login1>[, <login2>[, <login3>[...]]].
- Description セクション：目的と簡単な概要を含め、プロジェクトを明確に紹介する。
- Instructions セクション：コンパイル、インストール、実行に関する必要な情報を記載する。
- Resources セクション：関連する基本的な参考資料（ドキュメント、記事、チュートリアルなど）を挙げ、AI をどの作業・どの部分にどう使ったかを説明する。

➠ プロジェクトによっては、追加セクションが必要です（例えば使用例、機能一覧、技術選択など）。

必要な追加項目を以下に明記します。

- Configurationの節で、設定ファイルの構造と既定値を説明する。
- Highscoreの節で、ハイスコアシステムの仕組みと、その実装方法を選んだ理由を説明する。
- Maze Generationの節で、割り当てられたA-Maze-ingパッケージを迷路生成にどう使用するか説明する。
- Implementationの節で、実装の技術的な概要を説明する。
- General Software Architectureの節で、ソフトウェア設計の全体像（モジュール、クラス、それらの関係）を説明する。

- Project Managementの節で、プロジェクトの管理方法を簡潔に説明し、専用のプロジェクト管理ディレクトリへのリンクを示す。



README は英語で記述してください。

第X章

提出と相互レビュー

通常どおりGitリポジトリに課題を提出してください。口頭評価ではリポジトリ内の成果物だけをレビューします。ファイル名が正しいことを再確認してください。

再実装の指示

評価中に、短時間で行えるプロジェクトの変更を求められることがあります。小さな動作変更、数行のコードの記述や書き直し、簡単に追加できる機能などです。

すべてのプロジェクトで実施されるとは限りませんが、評価ガイドラインに記載されている場合には備えておかなければなりません。

この手順の目的は、プロジェクトの特定の部分を本当に理解しているかを確認することです。変更は、普段使う環境など、好きな開発環境で実施できます。評価で特に時間枠が定められていなければ、数分でできる内容であるはずです。
たとえば、関数やスクリプトの小さな更新、表示の変更、新しい情報を格納するためのデータ構造の調整などを求められる場合があります。

範囲や対象などの詳細は評価ガイドラインで指定され、同じプロジェクトでも評価によって異なる場合があります。